Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim: To obtain the classification of various classes using k-Nearest Neighbor approach**	
Experiment No: 03	**Date: 07/02/2026**	**Enrollment No: 92301733012**

Aim: To obtain the classification of various classes using k-Nearest Neighbor approach

IDE: Google Colab

Theory:

This algorithm is used to solve the classification model problems. K-nearest neighbor or K-NN algorithm basically creates an imaginary boundary to classify the data. When new data points come in, the algorithm will try to predict that to the nearest of the boundary line. Therefore, larger k value means smother curves of separation resulting in less complex models. Whereas, smaller k value tends to over fit the data and resulting in complex models. It's very important to have the right k-value when analyzing the dataset to avoid over fitting and under fitting of the dataset.

The model representation for KNN is the entire training dataset. It is as simple as that. KNN has no model other than storing the entire dataset, so there is no learning required. Efficient implementations can store the data using complex data structures like k-d trees to make look-up and matching of new patterns during prediction efficient. Because the entire training dataset is stored, you may want to think carefully about the consistency of your training data. It might be a good idea to curate it, update it often as new data becomes available and remove erroneous and outlier data.

K-NN algorithm assumes the similarity between the new case/data and available cases and put the new case into the category that is most similar to the available categories. K-NN algorithm stores all the available data and classifies a new data point based on the similarity. This means when new data appears then it can be easily classified into a well suite category by using K- NN algorithm. K-NN algorithm can be used for Regression as well as for Classification but mostly it is used for the Classification problems.

K-NN is a non-parametric algorithm, which means it does not make any assumption on underlying data. It is also called a lazy learner algorithm because it does not learn from the training set immediately instead it stores the dataset and at the time of classification, it performs an action on the dataset. KNN algorithm at the training phase just stores the dataset and when it gets new data, then it classifies that data into a category that is much similar to

the new data.

Example: Suppose, we have an image of a creature that looks similar to cat and dog, but we want to know either it is a cat or dog. So for this identification, we can use the KNN algorithm, as it works on a similarity measure. Our KNN model will find the similar features of the new data set to the cats and dogs images and based on the most similar features it will put it in either cat or dog category.

Methodology:

1. Load the basic libraries and packages
2. Load the dataset
3. Analyse the dataset
4. Pre-process the data

5. Visualize the Data
6. Separate the feature and prediction value columns
7. Select the number K of the neighbors
8. Calculate the Euclidean distance of K number of neighbors
9. Take the K nearest neighbors as per the calculated Euclidean distance.
10. Among these k neighbors, count the number of the data points in each category.
11. Assign the new data points to that category for which the number of the neighbor is maximum.
12. Our model is ready.

Pre Lab Exercise:

a. What is K in KNN-approach?

In the K-Nearest Neighbors (KNN) algorithm, K represents the number of nearest neighbors or data points the algorithm considers when making a prediction for a new, unclassified data point

b. Which are the types of distance metrices?

The types of distances metrices are:

- Euclidean Distance – Straight-line distance between two points (used for numerical data).
- Manhattan Distance – Sum of absolute differences (good for high-dimensional data).
- Minkowski Distance – General form of Euclidean and Manhattan (depends on parameter p).
- Chebyshev Distance – Maximum absolute difference between coordinates.
- Hamming Distance – Counts differences in binary/categorical data.
- Cosine Distance – Measures angle between vectors (commonly used in NLP).
- Jaccard Distance – Measures dissimilarity between sets.

c. What can be the criteria of selection of the value of K?

Choose the value of K by experimenting with multiple K values (for example, 1 to 20) and evaluating their performance using techniques like cross-validation.

Select the K that gives the highest accuracy (for classification) or lowest error (for regression) on validation data not just on training data.

The goal is to find a balance:

- Small K → very sensitive to noise, may cause overfitting (model memorizes data).
- Large K → smoother decision boundary, may cause underfitting (model becomes too general).

Therefore, the best K is the one that achieves strong validation performance while maintaining a good balance between bias and variance.

d. What are the advantages of KNN algorithm?

1. Simple and Easy to Understand
   KNN is very easy to implement and does not require complex mathematical calculations.
2. No Training Phase (Lazy Learning)
   It does not build a model during training, which makes it simple and flexible.
3. Works for Classification and Regression
   KNN can be used for both types of supervised learning problems.
4. No Assumption About Data Distribution
   It is a non-parametric algorithm, meaning it does not assume normal distribution of data.
5. Adaptable to New Data
   New data can be added easily without retraining the entire model.
6. Effective for Small Datasets
   Performs well when the dataset is small and not too noisy.

.

Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim: To obtain the classification of various classes using k-Nearest Neighbor approach**	
Experiment No: 03	**Date: 07/02/2026**	**Enrollment No: 92301733012**

Program (Code):

To be attached with

```
import time

import numpy as np

import pandas as pd

import matplotlib.pyplot as plt

import seaborn as sns


from sklearn.datasets import load_digits

from sklearn.model_selection import train_test_split, GridSearchCV

from sklearn.preprocessing import StandardScaler

from sklearn.pipeline import Pipeline


from sklearn.neighbors import KNeighborsClassifier

from sklearn.ensemble import RandomForestClassifier

from sklearn.neural_network import MLPClassifier


from sklearn.metrics import (

    accuracy_score,

    precision_score,

    recall_score,

    f1_score,

    confusion_matrix

)
```

```
digits = load_digits()
X = digits.data
y = digits.target
images = digits.images

plt.figure(figsize=(8, 4))
for i in range(10):
    plt.subplot(2, 5, i + 1)
    plt.imshow(images[i], cmap="gray")
    plt.title(f"{y[i]}")
    plt.axis("off")
plt.suptitle("Sample Digits")
plt.tight_layout()
plt.show()

X_train, X_test, y_train, y_test = train_test_split( X, y, test_size=0.2, random_state=42)

knn_pipeline = Pipeline([
    ("scaler", StandardScaler()),
    ("knn", KNeighborsClassifier())
])
knn_param_grid = {
    "knn__n_neighbors": [3, 5, 7, 9, 13, 15, 20]}
```

```
rf_pipeline = Pipeline([
    ("rf", RandomForestClassifier(random_state=42))
])
rf_param_grid = {
    "rf__n_estimators": [25, 50, 100, 200, 500],
    "rf__max_depth": [None, 10, 20]}
mlp_pipeline = Pipeline([
    ("scaler", StandardScaler()),
    ("mlp", MLPClassifier(
        max_iter=1000,
        random_state=42,
        early_stopping=True
    ))
])
mlp_param_grid = {
    "mlp__hidden_layer_sizes": [(64,), (128,), (64, 64)],
    "mlp__activation": ["relu", "tanh"],
    #"mlp__alpha": [0.0001, 0.001],
    "mlp__learning_rate_init": [0.001, 0.01, 1.0, 5.0, 10.0]
}
#Training the model
def train_with_timing(pipeline, param_grid):
    start = time.time()
    grid = GridSearchCV(
```

```
        pipeline,
        param_grid,
        cv=5,
        scoring="f1_weighted",
        n_jobs=-1
    )
    grid.fit(X_train, y_train)
    train_time = time.time() - start
    return grid, train_time
knn_grid, knn_train_time = train_with_timing(knn_pipeline, knn_param_grid)
rf_grid, rf_train_time = train_with_timing(rf_pipeline, rf_param_grid)
mlp_grid, mlp_train_time = train_with_timing(mlp_pipeline, mlp_param_grid)

#Inferencing the model
def inference_time(model):
    start = time.time()
    model.predict(X_test)
    return time.time() - start
knn_test_time = inference_time(knn_grid.best_estimator_)
rf_test_time = inference_time(rf_grid.best_estimator_)
mlp_test_time = inference_time(mlp_grid.best_estimator_)

#Evaluation
def evaluate_model(model):
    y_pred = model.predict(X_test)
```

```
    return {
        "Accuracy": accuracy_score(y_test, y_pred),
        "Precision": precision_score(y_test, y_pred, average="weighted"),
        "Recall": recall_score(y_test, y_pred, average="weighted"),
        "F1-score": f1_score(y_test, y_pred, average="weighted")
    }
knn_metrics = evaluate_model(knn_grid.best_estimator_)
rf_metrics = evaluate_model(rf_grid.best_estimator_)
mlp_metrics = evaluate_model(mlp_grid.best_estimator_)


#Result with comparison
results_df = pd.DataFrame(
    [knn_metrics, rf_metrics, mlp_metrics],
    index=["KNN", "Random Forest", "MLP"]
)
results_df["Training Time (s)"] = [
    knn_train_time, rf_train_time, mlp_train_time
]
results_df["Inference Time (s)"] = [
    knn_test_time, rf_test_time, mlp_test_time
]
print(results_df)
```

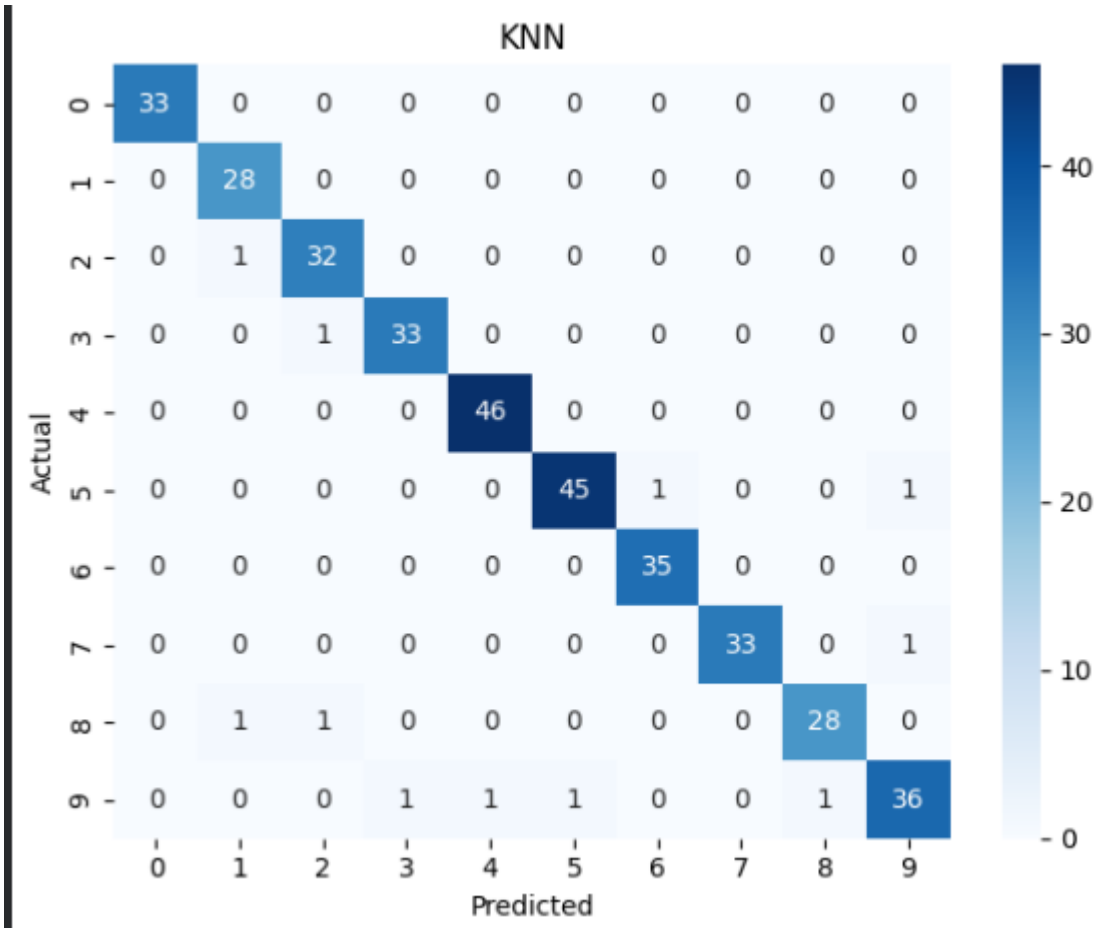
```
results_df.iloc[:, :4].plot(kind="bar", figsize=(10, 6))
plt.title("Model Performance Comparison (Digits)")
plt.ylabel("Score")
plt.ylim(0, 1)
plt.xticks(rotation=0)
plt.grid(axis="y")
plt.show()
models = {
    "KNN": knn_grid.best_estimator_,
    "Random Forest": rf_grid.best_estimator_,
    "MLP": mlp_grid.best_estimator_
}
fig, axes = plt.subplots(1, 3, figsize=(18, 5))
for ax, (name, model) in zip(axes, models.items()):
    cm = confusion_matrix(y_test, model.predict(X_test))
    sns.heatmap(cm, annot=True, fmt="d", cmap="Blues", ax=ax)
    ax.set_title(name)
    ax.set_xlabel("Predicted")
    ax.set_ylabel("Actual")
plt.tight_layout()
plt.show()
```

Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim: To obtain the classification of various classes using k-Nearest Neighbor approach**	
Experiment No: 03	**Date: 07/02/2026**	**Enrollment No: 92301733012**

Results:

To be attached with

a. Confusion matrix



b. Accuracy Score

```
...                 Accuracy  Precision    Recall  F1-score  Training Time (s)  \
    KNN             0.969444   0.969666  0.969444  0.969287           4.136447
    Random Forest   0.977778   0.978417  0.977778  0.977820          62.400273
    MLP             0.980556   0.980832  0.980556  0.980577          40.828275

                    Inference Time (s)
    KNN                      0.040893
    Random Forest            0.065224
    MLP                      0.002088
```

Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim: To obtain the classification of various classes using k-Nearest Neighbor approach**	
Experiment No: 03	**Date: 07/02/2026**	**Enrollment No: 92301733012**

Observation and Result Analysis:

a. Nature of the dataset

- ✓ The dataset is a supervised multi-class classification dataset.
- ✓ It contains 1797 handwritten digit images (0–9).
- ✓ Each image is 8×8 pixels, converted into 64 numerical features.
- ✓ All features are continuous pixel intensity values (0–16).
- ✓ The dataset is fairly balanced and suitable for testing classification algorithms.

b. During Training Process

- ✓ The dataset was split into 80% training and 20% testing.
- ✓ StandardScaler was applied for KNN and MLP.
- ✓ GridSearchCV with 5-fold cross-validation was used to find the best hyperparameters.
- ✓ The evaluation metric during training was F1-weighted score.
- ✓ Training time was recorded to compare computational efficiency.

Overall, hyperparameter tuning and cross-validation helped improve model performance and reduce overfitting.

c. After the training Process

After training, the best model from GridSearchCV for each algorithm (KNN, Random Forest, and MLP) was selected.

- ✓ The models were then evaluated on the test dataset (20%) using the following metrics:
- ✓ Accuracy
- ✓ Precision (weighted)
- ✓ Recall (weighted)
- ✓ F1-score (weighted)

Additionally:

- ✓ Inference time was measured to compare prediction speed.
- ✓ A bar chart was plotted to compare performance metrics.

Confusion matrices were generated to visualize classification performance and identify misclassifications.

d. Observation over the Learning Curve

From the learning behavior of the models:

- ✓ KNN tends to achieve high training accuracy quickly but may show slight performance variation depending on the value of K (small K → overfitting, large K → underfitting).
- ✓ Random Forest generally shows stable learning with good generalization and a small gap between training and validation performance.
- ✓ MLP improves gradually during training and, with early stopping enabled, prevents overfitting by stopping when validation performance stops improving.

Overall, the learning trend shows that proper hyperparameter tuning and cross-validation help maintain a good balance between bias and variance, resulting in strong generalization on the test data.

Post Lab Exercise:

a. Is KNN useful in regression-based problems?

Yes**,** KNN is useful for regression problems**.** In regression, the algorithm is called KNN Regressor. Instead of predicting the majority class (as in classification), it predicts the average (or weighted average) of the K nearest neighbors' target values.

b. Why KNN is a non-parametric algorithm?

Because, in regression tasks, the algorithm is known as KNN Regressor. Instead of predicting the majority class (as done in classification), it predicts a continuous value by calculating the average or weighted average of the target values of the K nearest neighbors. This allows KNN to model non-linear relationships by relying on the local patterns of nearby data point.

c. What are the assumptions of KNN approach?

KNN is a non-parametric algorithm, so it makes fewer assumptions compared to other models. However, it still relies on some important assumptions:

1. Similarity Assumption

   Data points that are close to each other in feature space are likely to have similar outputs (same class or similar target value).
2. Meaningful Distance Metric

   The chosen distance metric (e.g., Euclidean) correctly reflects similarity between data points.
3. Relevant Features

   All features contribute meaningfully to distance calculation (irrelevant features can reduce performance).
4. Proper Feature Scaling

   Features should be on a similar scale; otherwise, variables with larger ranges will dominate the distance.
5. Sufficient Data

   There is enough labelled data to represent the local structure of the problem.

d. How can the KNN approach make the prediction of the unseen dataset?

- ✓ Store Training Data

  KNN does not build a model during training. It simply stores all training data.
- ✓ Calculate Distance

  For a new (unseen) data point, it calculates the distance between the new point and all training points using a chosen distance metric (e.g., Euclidean distance).
- ✓ Select K Nearest Neighbours

  It selects the K closest data points based on the smallest distances.
- ✓ Make Prediction

For Classification: It predicts the class that appears most frequently among the K neighbours (majority voting).

For Regression: It predicts the average (or weighted average) of the neighbours' target values.

e. Why is KNN called a Lazy Learner?

KNN is called a Lazy Learner because it does not build a model during the training phase. Instead of learning patterns or creating a mathematical function in advance, KNN simply stores the entire training dataset**.** The actual computation happens only when a new (unseen) data point needs to be predicted.

f. Why should we not use KNN for large dataset?

KNN is generally not suitable for large datasets because:

1. High Prediction Time
   For every new data point, KNN calculates the distance to all training samples, which becomes very slow when the dataset is large.
2. High Memory Usage
   KNN stores the entire training dataset in memory, which requires large storage for big datasets.
3. Computationally Expensive
   Distance calculations in high-dimensional or large datasets increase computational cost significantly.
4. Scalability Issues
   As the dataset grows, both time complexity and memory requirements grow, making it inefficient.